



Faculté des Sciences et Technologies
Master 2 Ingénierie des Systèmes Complexes (ISC)

TD/TP

Sciences des données

Introduction aux réseaux de neurones

S. Barreto, P. Catala

Table des matières

1	Perceptron multi-couches	1
1.1	Modèle à deux couches	1
2	Customisation de l'architecture	2
2.1	Classification binaire	2
2.2	Classification multi-classes	2
3	Réseaux convolutionnels	2

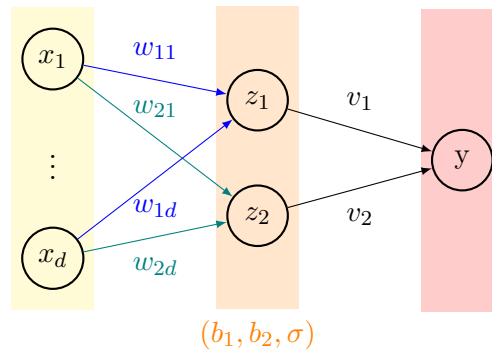
Un live script Matlab `sdd_tp3.mlx` avec l'énoncé est également fourni parmi les fichiers du tp.

1 Perceptron multi-couches

Dans cet exercice, nous allons entraîner un perceptron sur des données synthétiques. Pour cela, nous utiliserons les outils disponibles dans la toolbox "Deep Learning" de MATLAB.

1.1 Modèle à deux couches

1. On considère un perceptron à deux couches et de largeur 2 (*i.e.* avec deux neurones sur la couche cachée). Ce réseau est représenté ci-dessous :



Dans notre cas, $d = 2$; donner l'expression formelle de la sortie y en fonction

- du vecteur d'entrée $x := [x_1 \ x_2]^T$;
 - de la matrice $W := \begin{bmatrix} w_{11} & w_{12} \\ w_{21} & w_{22} \end{bmatrix}^T$ et du vecteur $v := [v_1 \ v_2]^T$ des poids ;
 - du vecteur de biais $b := [b_1 \ b_2]^T$;
 - de la fonction d'activation σ .
2. Utiliser la fonction Matlab `feedforwardnet` pour créer ce perceptron `nn` à une couche cachée avec deux neurones (on pensera à consulter l'aide de cette fonction).
 3. Les propriétés de la première (et unique) couche cachée sont enregistrées dans la variable `nn.layers{1}`. En particulier, la fonction d'activation pour cette couche est spécifiée par `nn.layers{1}.transferFcn`. Quelle est ici la fonction d'activation choisie par défaut ? On peut consulter la liste de toutes les fonctions d'activation possibles avec `help nntransfer`.
 4. Les algorithmes d'apprentissage partitionnent usuellement les données en plusieurs sous-ensembles (entraînement, validation et test) lors de l'entraînement. La stratégie de découpage est spécifiée dans `nn.divideFcn`, et l'ensemble des options s'obtient avec `help nndivision`. Ici, avec seulement 4 données, on n'effectue pas de partition.

```
nn.divideFcn = 'dividetrain'; % pas de partition des données
```

Parmi les autres options importantes, on notera :

- la fonction perte `nn.perfromFcn` (`help nnperformance`) ;
- l'algorithme d'optimisation `nn.trainFcn` (`help nntrain`), par exemple l'algorithme de descente de gradient ('`traingd`') et celui de Levenberg-Marquardt ('`trainlm`') utilisé par défaut ;
- les paramètre d'optimisations `nn.trainParam`. On pourra par exemple utiliser les réglages suivants :

```
nn.trainParam.epochs = 500; % nombre d'itérations
nn.trainParam.goal = 1e-6; % tolérance pour le critère d'arrêt
```

5. Afficher les sorties y obtenues pour différents vecteurs d'entrée x avec le réseau de neurones. Pour retourner une valeur binaire, on peut par exemple arrondir le résultat.

2 Customisation de l'architecture

Cet exercice vise à vous faire observer l'impact de l'architecture sur les résultats de classification.

2.1 Classification binaire

1. Utiliser la fonction matlab fournie `data2class` pour générer un jeu de données bi-classes, avec 200 échantillons par classe. Afficher le jeu de données sur le domaine $[-1, 1] \times [-1, 1]$ (avec une couleur par classe).
2. Entraîner un réseau de neurones `nn_trained` à une couche cachée avec 3 neurones sur ces données.
3. Faire un nouveau tirage des données, et les afficher en même temps que la fonction `nn_trained` apprise. Pour évaluer `nn_trained` sur l'ensemble du domaine, on pourra s'aider du code suivant.

```
t = linspace(-1,1,100);
[B,A] = meshgrid(t);
D = [A(:) B(:)]';
nn_output = round(nn_trained(D));
nn_output = reshape(F,[201,201]);
```

4. Tester plusieurs architectures (nombre de couches, largeur, fonctions d'activation)

2.2 Classification multi-classes

1. Tester de même plusieurs architectures sur les données générées avec `data3class` et afficher les résultats comme précédemment. A ce stade vous pouvez aussi essayer de changer la fonction objectif utilisée par le réseau, et spécifiée par `nn.performFcn`. Par défaut, Matlab choisit l'erreur quadratique moyenne ('mse'). La liste des choix possibles est donnée par `help nnperformance`.

3 Réseaux convolutionnels

Dans cet exercice, nous allons construire et entraîner un réseau de neurones convolutionnel simple, sur des images extraites du jeu de donnée CIFAR-10. Ces données sont fournies dans les fichiers du tp. Elles consistent en des images de taille 32×32 .

Charger les images du jeu d'entraînement à l'aide des commandes suivantes

```
categories = {'cat', 'dog'}
img_train = imageDataStore(fullfile('cifar10train',categories),LabelSource='
    foldernames');
```

La variable `img` est une structure contenant les chemins vers les images (`img.Files`), et plusieurs informations utiles comme les labels (`img.Labels`). Vous pouvez changer les classes utilisées si vous le souhaitez, ou en rajouter.

1. On peut charger une image dans une matrice à l'aide de la commande `imread`.

```
I = imread(img.Files{i});
```

Afficher quelques images du jeu de données à l'aide de `imagesc`.

2. On peut charger toutes les images et leurs labels à l'aide de

```
data_train = readall(img_train);
labels_train = img_train.Labels;
```

La variable `data_train` se présente sous la forme d’une cellule Matlab. Stockez-la dans un unique de tenseur `Xtrain` de dimension $32 \times 32 \times 3 \times (\text{nombre d'images})$.

Les couches latentes d’un réseau de neurones convolutionnels alternent généralement entre

- des couches de convolution, que l’on initialisera avec la commande `convolution2dlayer(..., 'Padding', 'same')`. Il faudra spécifier la taille des filtres (en pixels), ainsi que leur nombre;
- des couches de normalisation `batchNormalizationLayer`, optionnelle pour stabiliser l’apprentissage;
- des couches de regroupement, ou ”pooling”, où la dimension des données est réduite en remplaçant les pixels d’une région par un pixel avec la valeur maximum sur la région (`maxPooling2dLayer(...)`), ou bien la valeur moyenne (`averagePooling2dLayer(...)`). L’argument nécessaire dans les deux cas est la taille des régions;
- des couches d’activation (comme `reluLayer` ou `softmaxLayer`);
- des couches complètement connectées (`fullyConnectedLayer(...)`), où la taille des données en sortie doit être spécifiée.

Dans le cas d’images, la première couche contenant les données se déclare avec `imageInputLayer(size)`, où `size` représente la taille des images.

Les deux dernières couches sont usuellement une couche complètement connectée, avec comme taille de sortie le nombre de classes, suivie d’une activation softmax.

Pour des tâches de classification, la couche de sortie se déclare par `classificationLayer`.

La syntaxe Matlab pour déclarer une architecture est la suivante :

```
layer = [
    couche1
    couche2
    ...
    coucheN];
```

2. Construire une première architecture simple, constituée : d’une couche d’entrée, une couche de convolution (taille 5, quantité 32), une couche de regroupement par maximum (taille 5), une couche Relu, une couche complètement connectée (le nombre de classes en taille de sortie), une couche softmax.
3. Entraîner ce réseau à l’aide de la commande `trainnet`. On spécifiera les options suivantes pour l’optimisation.

```
options = trainingOptions('sgdm', ... % descente de gradient stochastique
    InitialLearnRate= 0.001, ... % pas de descente
    LearnRateDropFactor= 0.1, ...
    LearnRateDropPeriod= 8, ...
    L2Regularization= 0.004, ...
    MaxEpochs= 10, ...
    MiniBatchSize= 100, ...
    Verbose= true);
```

4. Charger les images test dans un tenseur `X_test` comme en 2. La commande `predict(cnet,Xtest)` retourne, pour chaque échantillon test, un vecteur de probabilité d’appartenance aux différentes classes.

Déterminer le résultat de la classification sur les échantillons tests, en supposant que l’on choisit la classe ayant la probabilité maximale.

5. Afficher quelques images du jeu de test, en indiquant en titre la classification obtenue.
6. Afficher la matrice de confusion (`confusionchart`).
7. Tester d'autres architectures de réseau, par exemple en rajoutant des couches que vous choisirez et initialiserez.